Source-pinned technical review

AsymptoticAnalysis

New findings at the observable and native-backend boundaries

Mathematical correctness, Wolfram Language comparison, evaluation semantics, focused repairs, and development priorities

Prepared for Vladimir Reshetnikov
9 September 2026

Repository

https://github.com/VladimirReshetnikov/Asymptotic

Reviewed revision

6687962f3c858a4f93623cfc496f33e35c6763d4

Commit time: 9 September 2026, 17:18:17 Pacific daylight time.

Evidence. Public wrong results were reproduced on Wolfram Language 15.0.0 for Linux. Three focused controls were also run against a temporary patched standalone. Fourteen independent Python tests passed. These are scoped observations, not a full upstream-suite run or acceptance of every supplied regression and patch artifact.

Abstract

The current repository combines a real asymptotic-germ calculus with a newer native delegation layer. Its contribution is not simply another implementation of Taylor expansion or series reversion: it retains exact-weight blocks, coordinates, branches, error descriptors, operation recipes, and specialized inverse cores in an inspectable result object. The native layer appropriately preserves Wolfram results without promoting their formal order to a proved analytic error bound.

This review concentrates on additions to the eighteen existing review packages. Two mathematical failures were reproduced in the generic observable path. First, the one-sided constant coefficient returned by native expansion is replaced by the observable's value at the endpoint. For `FractionalPart`, this loses an entire constant term. Second, a native Taylor result of lower order than requested is read as though its unknown coefficients were zero, producing an unjustified higher-order remainder. A third reproduced defect makes the backend dispatcher ignore the canonical constructor's configured backend default, changing both order conventions and package-only admission policy. Smaller findings concern zero-order differentiation with a cutoff, unknown symbolic options, and eager native-result normalization.

Each finding is classified against the existing register. The article develops the needed sided-germ and native-oracle contracts, compares the package fairly with Wolfram Language, supplies conservative candidate edits and reproduction programs, and proposes a narrowly targeted development sequence. It does not repackage the already documented dense-export, assumption-capture, certificate, flat-grade, or general transseries-roadmap findings as new discoveries.

Contents

1 Assessment and scope

1.1 The most important conclusion

The next correctness improvement should be made at the boundary where a *native coefficient oracle becomes a package analytic observable*, rather than by adding another special-function family. That boundary currently mixes three different pieces of information: the point value of a function, its one-sided limiting germ, and the precision actually returned by its Taylor oracle. Two successful package results show that these distinctions are not merely theoretical. [4, 7]

The native dispatcher has a different but equally concrete contract error: `SetOptions[AsymptoticExpansion, "Backend"->...]` changes the public option table but not the no-selector dispatch path. Because package and native orders have intentionally different meanings, ignoring this default changes the mathematical object returned, not just a descriptive label. [6]

The repository nevertheless has several important safeguards worth preserving. Native results carry `Missing["NativeContract"]`, not a fabricated `PowerLogRemainder`. Analytic operations explicitly reject those results. The interval-certificate implementation verifies source domains over a closed interval and distinguishes a root enclosure from an asymptotic approximation. The recent certificate adaptation work is present; older reports of that particular stagnation should not be repeated as current defects. [7, 8, 10]

1.2 Findings at a glance

The identifiers N01–N06 in this article are local identifiers. Upstream `C06`, `C13`, and similar identifiers refer to the maintained review register, not to the numbering of any one earlier article.

ID	Priority	Finding	Evidence / novelty
N01	High	Observable reconstruction replaces a one-sided Taylor constant by an endpoint value; a pure remainder can also lose the needed approach side.	Two public wrong results; sharper C13/C16 delta.
N02	High	Generic observable import reads beyond the native result's known order, treating unavailable coefficients as zero.	Truthful shortened-oracle witness; new path-specific C06 delta.
N03	High	The canonical constructor's backend default is ignored, including a package-only default.	Two native default-versus-explicit contrasts; post-review dispatcher defect.
N04	Low	Zero-order differentiation returns the input unchanged despite an explicit cutoff.	Native observation; API policy/consistency issue, not a false asymptotic formula.
N05	Medium	An unknown symbolic option is classified as a specification and escapes the package's exclusive-option check.	Warnings followed by a successful result; request-grammar issue.
N06	Opportunity	Native wrapping eagerly computes `Normal`, even when only the native payload is needed.	One observed extra callback; no timing or memory-speedup claim.

N01 and N02 are not represented as discoveries that analyticity or precision matters in general. Earlier reviews

already identify those obligations. The increment here is a concrete built-in discontinuity witness, an independently identifiable loss of the native constant coefficient, and a still-unchecked sibling import path after precision checks were added elsewhere. N03 and N05 arise in the newer native dispatcher. The distinction matters for deciding which existing register entries to extend and which new entries to create.

1.3 Snapshot and source coverage

The analysis is pinned to `6687962f3c858a4f93623cfc496f33e35c6763d4`. The package context and root standalone are now `AsymptoticAnalysis‘` and `AsymptoticAnalysis.wl`. Older reports refer to older contexts and directory layouts; their source paths should not be copied directly into current patches. The main modular loader supplies the current module ordering. [2, 1]

Close source inspection covered the native dispatcher, ordinary series operations and arithmetic, inverse-function expression entry, exact interval certificates, central public definitions, and the ordered Gamma/Barnes inverse constructor. The documentation, review indices, and consolidated finding crosswalk supplied the broader feature and suppression baseline. This is not a claim that every kernel module, notebook, historical test, or proof received a line-by-line audit. Appendix B and the machine-readable coverage record state the limits.

The existing review directory contains eighteen packages, consolidated into 123 named finding entries in the maintained crosswalk. That register was read as the primary deduplication source. Selected original discussions were also read, and the eighteen root article TeX files were searched for terminology specific to the new findings. Some root articles include other TeX files, so that lexical scan is supplementary evidence, not a claim of full-corpus semantic comparison. [11, 10, 12, 13]

1.4 Execution boundaries

The successful native observations used

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

with the repository-root standalone downloaded from the pinned raw URL and loaded with `Get[URLDownloa d[...]]`. They do not establish behavior on every kernel version or operating system. Several other connector calls failed with network errors or HTTP 502; no results are inferred from those calls.

The bundled `native_observations.json` is a transcription of the returned observations, not a fabricated `TestReport`. The independent Python checks use exact rational arithmetic and source-anchor fixtures. They check the counterexample mathematics and patch tooling, not execution of the Wolfram package. The complete supplied WLT files were not run as a suite. These limits are relevant when deciding whether to merge the proposed edits.

2 Architecture and mathematical meaning

2.1 A family of cooperating representations

The public interface is a common wrapper around several partially overlapping representations, not one unrestricted transseries field. For an ordinary power–log expansion, write the positive local coordinate as $w \to 0^+$ and let

$$f(w) = \sum_{\beta \in B} w^{\beta} P_{\beta}(\log w) + O\Big(w^{\rho}(1 + |\log w|)^{k}\Big). \tag{1}$$

The exponents are exact real values; B is a finite ordered support and the coefficients are logarithmic polynomials. The ordinary jet carries rows, a remainder exponent, and a logarithmic degree. Its validity is an analytic statement on a positive real approach, not just a list of coefficients. [2, 4]

The explicit operation layer generalizes this to

$$b(x) + A(x)\big(J(w(x)) + O(R(w(x)))\big), \tag{2}$$

with exact offset b and prefactor A. The absolute error contains $|A|$. Consequently a cutoff inside a factored bracket cannot automatically be compared with an absolute cutoff in another representation. The code's `seriesData`, `seriesFlat`, and `seriesMake` functions are central places where that distinction is carried across operations. [4]

The documented specialized representations include logarithmic inverse scales, ordered Gamma/Barnes inverse corrections, finite flat sectors, Fourier-type oscillatory coefficients, and composite error envelopes when a shared ordered algebra is unavailable. A composite envelope is useful precisely because it can retain a sound absolute-error statement without pretending that all terms belong to one scalar exponent lattice. This existing design should not be replaced by an unqualified claim that every successful result has the same notion of order. [1, 3, 5]

2.2 Inverse engines and exact cores

A common ordinary inverse model is

$$y - y_0 = au^p \left(1 + \sum_j u^{\delta_j} B_j(\log u)\right), \qquad u > 0, \quad \delta_j > 0. \tag{3}$$

The package exposes Lagrange, grouped-Lagrange, and Newton-oriented routes, retains inverse provenance, and supports refinement and residual inspection. The positive coordinate and selected source side are indispensable when p or the correction weights are nonintegral. The model's multi-index structure is not the same thing as a dense rational-power `SeriesData` lattice. [2, 3, 1]

For a simple illustration of the supported mathematical shape, consider $y = x + cx^{1+\delta}$ with $x \to 0^+$ and fixed $\delta > 0$. Seeking $x = yH(cy^\delta)$ reduces the inverse to an ordinary implicit analytic equation for H near one. Its first terms are

$$x = y - cy^{1+\delta} + (1+\delta)c^2y^{1+2\delta} - \frac{(1+\delta)(2+3\delta)}{2}c^3y^{1+3\delta} + \cdots. \tag{4}$$

Substitution verifies these coefficients directly. When δ is irrational, the weights still form a simple ordered family, even though they are not an integer-indexed rational Puiseux lattice. This example explains the value of explicit weight metadata; it is not a claim that no native Wolfram expression or asymptotic solver can produce the same formula.

For inverse Gamma, the implementation extracts an exact Lambert core from the leading logarithmic equation. If T denotes the transformed target, then

$$X(\log X - 1) = T, \qquad X = \frac{T}{W(T/e)}. \tag{5}$$

Indeed, putting $W = W(T/e)$ gives $T = eWe^W$, $X = e^{W+1}$, and the identity follows. The correction engine uses $t = 1/X$ and $q = 1/\log X$ and retains a complete polynomial in q at each power of t. The corresponding Barnes core in the code is $X = \sqrt{4T/W(4T/e^3)}$, with coefficient variable $1/(\log X - 1)$. These are exact cores for leading equations, not exact inverses of the full special functions. [9]

The finite Stirling and Barnes logarithmic expansions used by these engines are Poincare expansions with finite-order remainder obligations. The source correctly records that the forward asymptotic series is not being asserted convergent. The relevant classical expansions are documented in DLMF Sections 5.11 and 5.17. [28, 29, 9]

2.3 Three distinct kinds of evidence

An ordinary analytic expansion, a native formal result, and an interval certificate answer different questions. In an analytic expansion, a remainder class describes a family of possible errors near a selected limit. A native result preserves what the selected Wolfram function returned; it may contain `SeriesData`, conditions, lists, an ordinary expression, or an unresolved call. An interval certificate encloses a particular root on a specified finite interval using explicit arithmetic and domain checks. [7, 8]

The certificate code evaluates rational intervals with outward dyadic rounding, explicit exponential/logarithm tails, a derivative enclosure separated from zero, and either residual-bracket containment or endpoint signs. Its sharp root enclosure is distinct from the initial symmetric residual radius. It also states that unspecified input-remainder families are not automatically covered by a certificate for the retained explicit function. These are substantial contracts; they should not be conflated with the quality of a numerical root seed. [8]

The current native wrapper explicitly sets

```
"Kind" -> "Native"
"Remainder" -> Missing["NativeContract"]
"Exact" -> Missing["NotEstablished"]
```

and rejects package analytic arithmetic through `requireAnalyticSeries`. The new findings do not invalidate that separation. N02 is a different route: a native result is consumed *inside* an analytic observable constructor, where the outer Native wrapper is never created. A boundary guard in the public dispatcher therefore cannot repair it. [6, 4]

3 Comparison with Wolfram Language

3.1 A fair baseline

It would be incorrect to contrast this package with a Wolfram system limited to ordinary Taylor series. Native `Series` supports several non-Taylor forms, while `Asymptotic` constructs approximations in broader asymptotic scales. Native inverse and equation-solving facilities also overlap materially with the repository. The differentiator is the package's particular persistent real-germ representation and the operations and evidence retained with it, not exclusive ownership of asymptotic mathematics. [14, 15, 19]

Capability	Wolfram Language baseline	Package specialization / present boundary
Local expansion	`Series` handles Taylor, Laurent, selected fractional-power and logarithmic expansions, infinity, and successive variables.	Exact real-weight power–log blocks and an explicit positive-real local coordinate; specialized carriers can remain exact. [14, 2]
General asymptotics	`Asymptotic` supports more general scales and condition/direction controls.	Package results retain explicit error descriptors and operation provenance; native delegation preserves the native contract separately. [15, 7]
Formal reversion	`InverseSeries` and `ComposeSeries` operate on native series representations.	Real branch/source metadata, multiple inverse engines, exact inverse cores, and inspectable residual/refinement facilities. [17, 18, 2]

Capability	Wolfram Language baseline	Package specialization / present boundary
Implicit solutions	`AsymptoticSolve` handles scalar and multivariate equations; an option can request native series output for appropriate solutions.	Focused scalar inverse constructions with package-specific complete-block and branch contracts; no universal superiority over native solving is established. [19, 1]
Series representation	`SeriesData` records a coefficient lattice and formal order.	Sparse real weights, polynomial logarithmic coefficients, factored and special inverse scales; conversion is not universally lossless. [16, 4, 9]
Algebra and composition	Native series support arithmetic and composition; `Normal` deliberately removes special structure.	Error transport and retained recipes across admitted analytic operations. The generic observable boundary has N01/N02 defects. [18, 20, 4]
Domains and regularity	`FunctionDomain` and `FunctionAnalytic` provide symbolic domain/regularity analysis under documented assumptions.	Branch-aware admission and stored domains, but a native coefficient table does not itself prove the local regularity needed by every consumer. [24, 23, 4]
Asymptotic comparison	`AsymptoticLess` supplies a little-o comparison facility.	Remainder metadata supports subsequent precision planning; comparison predicates are useful independent oracles, not a substitute for transporting that metadata. [25, 4]
Integrals and sums	`AsymptoticIntegrate` and `AsymptoticSum` address parameter-dependent integration and summation.	They are not replaced by the package's inverse-germ calculus. General adapters and integration extensions are already upstream roadmap items, not new findings here. [26, 27, 10]
Pointwise verification	Native numerical and symbolic tools can support checking and enclosure workflows.	`InverseCertificate` implements a narrowly specified exact rational enclosure pipeline for supported explicit functions. [8]

3.2 Order conventions must be compared before coefficients

For the observed call with `Exp[x]` and `x,0,2`, the package route returns $1 + x$, whereas explicit native `Series` mode returns $1 + x + x^2/2$ after normalization. Both are consistent with their intended order conventions: the package cutoff is exclusive, and native `Series` includes the requested power. The N03 defect is that a user-selected default for the second behavior still produces the first. The difference is therefore not a mathematical disagreement between engines; it is a dispatch error. [7, 14]

A serious differential comparison should match achieved information, not merely pass the same integer to two APIs. For an ordinary integer-power example, converting a package cutoff into a native requested degree may be easy. For fractional weights, logarithmic blocks, extracted prefactors, or successive variables, a universal "add one" or "subtract one" rule is not sufficient. That distinction is already correctly described in the repository's native contract note. [7]

The test harness should retain the native result, its order convention, and the package's achieved error class before comparing finite expressions. Two identical finite expressions can have different valid error guarantees; two different expressions can both be correct at different orders. Comparison of `Normal` alone is especially weak for

detecting N02, where the missing cubic coefficient is hidden behind a false higher-order claim.

3.3 Native breadth and package closure are separate axes

Explicit native delegation substantially broadens the public constructor's input range. It does not make every returned object eligible for the package's analytic calculus. This is a sensible split: a complex or multivariate native result may be perfectly meaningful while failing the premises of a positive-real scalar remainder theorem. [6, 7]

Automatic native coverage is still a maintained work item in the existing register. This article does not relabel that known limitation as a fresh finding. It instead identifies two additional obligations within that work: backend defaults must really control selection, and syntactic role discovery must not depend on an option name accidentally being present in an option table. Those obligations can be checked without claiming a theorem that the wrapper covers every native input. [10, 6]

3.4 What an analyticity check can and cannot replace

`FunctionAnalytic` can provide useful sufficient evidence on a specified neighborhood. Its documented domain and conditional-result conventions still need to be respected; a returned condition is not an unconditional proof. Likewise, `FunctionDomain` describes admissibility, not the Taylor remainder order of an arbitrary opaque function. [23, 24]

For the present observable bug, a more elementary distinction is decisive: *a one-sided polynomial expansion can be valid even when the function's value at the endpoint differs from its limiting constant*. That is why changing a generic `Series` call to `Analytic->False` cannot by itself repair N01. The native oracle can return a correct one-sided expansion, and the package can corrupt it afterwards. Section 4 demonstrates exactly that sequence.

4 N01: endpoint values are not one-sided germ constants

Priority: high. Native public wrong results reproduced.
Source: `seriesJetApply`, generic unary-observable branch in `src/Kernel/SeriesOperations.wl`.
Novelty: a concrete built-in witness and reconstruction defect extending C13/C16, not a repetition of the general need for regularity evidence.

4.1 Minimal public witness

After loading the pinned package, evaluate

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  1-x, {x,0,1}, "Backend"->"Package"];
r = AsymptoticAnalysis`SeriesObservable[s, FractionalPart[z], z];
{r["Expression"], r["Remainder"]}
```

The observed output was

```
{0, PowerLogRemainder[x,1,0]}
```

The input object retains $1 + O(x)$, with source function $1 - x$. On the requested positive approach, however,

$$\text{FractionalPart}(1 - x) = 1 - x, \qquad 0 < x < 1. \tag{6}$$

Thus the actual function tends to one, whereas the returned finite expression plus the claimed error tends to zero. No choice of the implicit Big-O constant can repair the statement. The elementary fractional-part identity is also consistent with its documented definition. [22]

For an exact rational diagnostic sequence $x_n = 1/n$,

$$\frac{\text{FractionalPart}(1 - x_n) - 0}{x_n} = n - 1 \longrightarrow \infty. \tag{7}$$

This is an asymptotic contradiction, not merely a large numerical discrepancy at a point far from the limit. The independent checks verify the exact ratios for a range of rational inputs; the displayed formula proves the divergence for the entire sequence.

4.2 Retaining the missing direction does not cure the bug

A tempting diagnosis is that the first call discarded $-x$, leaving an insufficiently specified approach to the discontinuity. That is one part of the problem, but not the whole problem. Repeating the call with cutoff two retains the negative deviation:

```
s = AsymptoticAnalysis‘AsymptoticExpansion[
  1-x, {x,0,2}, "Backend"->"Package"];
r = AsymptoticAnalysis‘SeriesObservable[s, FractionalPart[z], z];
{r["Expression"], r["Remainder"]}
(* observed: {-x, PowerLogRemainder[x,2,0]} *)
```

The true value is again $1 - x$. The returned expression has lost exactly one, and the claimed error is now even smaller:

$$\frac{(1 - x) - (-x)}{x^2} = x^{-2} \longrightarrow \infty. \tag{8}$$

Therefore source refinement or preservation of the first omitted term is not, by itself, a sufficient repair.

4.3 The native oracle is correct in this example

A direct native control was executed with analytic assumptions disabled:

```
Series[FractionalPart[1-u], {u,0,2},
  Assumptions->u>0, Analytic->False]
(* SeriesData[u,0,{1,-1},0,3,1] *)

Series[FractionalPart[1+u], {u,0,2},
  Assumptions->u>0, Analytic->False]
(* SeriesData[u,0,{1},1,3,1] *)

FractionalPart[1]
(* 0 *)
```

The left-approach constant is one; the right-approach constant and the point value are zero. The direct native call returned these objects with internal `MinValue`/`MaxValue` messages, which are recorded in the evidence. The objects themselves distinguish the two sides correctly.

This sharpens the earlier recommendation to consider `Analytic->False` at generic import points. That option may help with an opaque-function regularity problem, but it cannot repair a consumer that discards a correct native constant. The present example uses a built-in function and survives that proposed oracle-side change. [13, 4]

4.4 Source-level mechanism

The generic observable path computes the input jet, extracts a finite limiting constant c, chooses a sign for the small variable, and requests

```
native = Quiet[Series[h[c + sign u], {u,0,n},
  Assumptions -> ass && u > 0]];
```

It constructs a coefficient accessor for `native`, but the final constant term is not taken from that accessor. Instead it uses

```
pAdd[pConst[h[c], ell, ass], res, ell, ass]
```

and the pure-remainder branch similarly returns a constant built from `h[c]`. For h = FractionalPart and $c = 1$, the left-sided native constant one is replaced by the point value zero. [4]

When no small retained block exists, the code initially chooses the positive side. A statement $g = 1 + O(x)$ does not determine the sign of $g - 1$. It includes $g = 1 - x$, $g = 1 + x$, the constant function one, and functions crossing the endpoint arbitrarily often. One-sided native data cannot be applied to that entire family without additional information. This is the distinct uncertainty-side part of N01.

4.5 A precise germ invariant

Proposition 4.1 (Endpoint-value invariance under separated composition)**.** *Let $g(w) \to c$ as $w \to 0^+$, and suppose $g(w) \neq c$ for every sufficiently small positive w. Let H_1 and H_2 agree on a punctured neighborhood of c, but possibly differ at c. Then $H_1(g(w))$ and $H_2(g(w))$ agree for all sufficiently small positive w.*

Proof. For sufficiently small w, the value $g(w)$ lies in that punctured neighborhood. The equality follows by substitution. □

This gives a useful test property: changing only an outer function's point value must not change the result when the inner germ is eventually separated from that point. The separation premise must not be omitted. For an exactly constant inner function, or one that reaches c arbitrarily close to the limit, the point value can genuinely affect composition. An implementation therefore needs more than the Boolean fact that an input tends to c.

4.6 The complete repair has four admission cases

An exact constant inner germ should be evaluated at the actual point value. An inner germ entering a two-sided neighborhood where the observable has a sufficiently controlled Taylor expansion can use the usual analytic calculus. An eventually one-sided inner germ can instead use a sided analytic extension, whose constant need not equal the endpoint value. Finally, an input with an undetermined side must not be fed to a sided oracle unless both possible sides satisfy a compatible contract.

For the sided case, write

$$H(c+\sigma v)=\sum_{j=0}^{M-1}a_j v^j+O(v^M),\qquad v\to 0^+,\quad \sigma\in\{-1,1\}. \tag{9}$$

The zeroth coefficient is

$$a_0=\lim_{v\to 0^+}H(c+\sigma v), \tag{10}$$

not automatically $H(c)$. If $g(w)=c+\sigma v(w)$ and $v(w)>0$ eventually, substitution into (9) is the appropriate operation. Sign information must apply to the entire precision-tracked inner germ, not just a displayed constant or a heuristic sample.

The implementation can represent this with a small observable-admission record:

```
<|"Center" -> c,
  "Approach" -> "ExactPoint" | "TwoSided" | "FromAbove" | "FromBelow",
  "GermConstant" -> a0,
  "PointValue" -> h[c],
  "NativeExclusiveOrder" -> M,
  "RegularityEvidence" -> evidence,
  "SideEvidence" -> sideEvidence|>
```

These field names are a design proposal. The important change is to prevent a point value from being substituted for a germ coefficient by convenience. This is substantially narrower than a general scale-registry redesign.

4.7 Conservative candidate patch

The supplied interim patch does not implement all four cases. It screens generic nonconstant observable inputs for two-sided continuity at the limiting argument, using two bounded native limits and explicit comparison with $H(c)$. If the screen cannot prove agreement, it returns `Failure["UnprovedObservableContinuity",...]`.

This deliberately refuses some valid one-sided requests. In particular, it rejects the known left-approach `FractionalPart` example instead of computing its perfectly valid sided expansion. Such a refusal is preferable to a false analytic error bound, but it is not the final desired feature set. Exact constant inputs are exempt from the new continuity screen.

Continuity alone is not a complete Taylor-remainder theorem. The patch is a necessary-condition hardening measure addressing the reproduced discontinuity failure; it does not close all of upstream C16. A complete repair still needs the differentiability or analytic-extension evidence appropriate to the requested order. The native prototype rejected the retained-left witness and preserved $x-x^3/6+O(x^5)$ for the ordinary `Sin` control.

5 N02: a short native jet is read as a longer one

Priority: high at the native-oracle boundary. Reproduced with a truthful user-defined Series handler.
Source: the same generic branch of `seriesJetApply`.
Novelty: the missing order check is in a sibling path not covered by the native-import repair recorded as C06. This is not a claim that built-in `Sin` itself returns the shortened jet used in the test.

5.1 A lawful shortened oracle

The following handler represents the ordinary analytic sine function on numeric arguments and deliberately returns a valid but coarse native expansion:

```
ClearAll[shortSin];
shortSin[0] = 0;
shortSin[t_?NumericQ] := Sin[t];
shortSin /: Series[shortSin[v_], {u_Symbol,0,n_Integer}, opts___] /;
    v === u := SeriesData[u,0,{1},1,2,1];
```

The reported series is $u + O(u^2)$. This is truthful for $\sin u$: its actual error is of order u^3, which is also $O(u^2)$ near zero. No false derivative, false coefficient, or forged package result is supplied. The boundary is being tested with an oracle that does less work than requested but accurately reports what it knows.

Now evaluate

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  x, {x,0,5}, "Backend"->"Package"];
r = AsymptoticAnalysis`SeriesObservable[
  s, shortSin[z], z, "Cutoff"->5];
{r["Expression"], r["Remainder"]}
(* observed: {x, PowerLogRemainder[x,5,0]} *)
```

The package has promoted a known native cutoff of two to an analytic cutoff of five. Since

$$\sin x - x = -\frac{x^3}{6} + O(x^5), \qquad \frac{\sin x - x}{x^5} \longrightarrow -\infty, \tag{11}$$

its returned bound is false. The displayed limit was independently evaluated in the native control call. An entirely rational polynomial witness, $H(x) = x - x^3/6$, proves the same precision contradiction without trigonometry.

5.2 Where the unavailable coefficients become zeros

After the native result passes structural checks, the accessor is essentially

```
cf[k_] := If[k >= native[[4]] &&
    k-native[[4]]+1 <= Length[native[[3]]],
  native[[3,k-native[[4]]+1]], 0]
```

The structural checks ensure an ordinary nonnegative integral-power native series and coefficients independent of the dummy variable. They do not verify that `native[[5]]`, the native exclusive order numerator in this admitted unit-denominator case, reaches the order required by the composition. [4, 16]

There are two different reasons a coefficient might be absent from the stored list. It can be a known zero *below* the native order boundary, or it can be unavailable *at or beyond* that boundary. The accessor conflates them. The requested order of the `Series` call is not a substitute for inspecting the order of the expression actually returned.

This bug does not require the native backend to lie. A partial computation, a custom truthful `Series` handler, or a future native result shape with a shorter certified prefix is enough. Treating a service's request as its response postcondition is unsafe at a mathematical boundary just as it is at an ordinary streaming or parsing boundary.

5.3 The correct transported precision

Suppose a native oracle supplies

$$H(c+v) = \sum_{j=0}^{M-1} a_j v^j + O(v^M) \tag{12}$$

and the inner coordinate satisfies

$$v(w) = O\Big(w^{\alpha}(1+|\log w|)^{d}\Big), \qquad \alpha > 0. \tag{13}$$

Then the oracle's omitted part contributes at most

$$O\Big(w^{\alpha M}(1+|\log w|)^{dM}\Big). \tag{14}$$

There may also be an error propagated from the uncertainty in v itself. Both contributions must be combined with the finite truncation frontier. It is not valid to retain only the inner error and the caller's requested cutoff.

For the witness, $\alpha = 1$, $d = 0$, and $M = 2$. The available outer-oracle evidence supports an $O(x^2)$ cap, not $O(x^5)$. The actual sine function happens to satisfy a stronger cubic error, but that extra fact was not returned by the handler and cannot be inferred by declaring unknown coefficients zero.

A production importer can either return a sound coarser result with explicit achieved-order metadata, ask the oracle for more information under a bounded policy, or refuse the requested operation. It must not silently strengthen the error class. The same distinction applies when the first omitted native term has logarithmic structure; the exponent alone is then insufficient to describe the transported boundary.

5.4 The supplied order guard

The candidate patch adds a check before the coefficient accessor is used:

```
If[! TrueQ[native[[5]] > n],
  fail["InsufficientNativeObservablePrecision", ...]];
```

Here n is the integral coefficient order requested by this particular generic observable branch, and the preceding admission check already requires native denominator one. Requiring the returned exclusive order to exceed n is a simple conservative postcondition. It is stricter than a fully optimized precision-transport calculation in some boundary cases, but it cannot invent missing coefficients.

A complete future importer should expose a three-way coefficient query: `Known[value]`, `KnownZero`, or `BeyondPrecision`. At present the last two are represented by the same scalar zero. Centralizing that distinction would make it harder for later adapters to repeat the same error under a different function name.

5.5 How this relates to the existing C06 repair

The register documents native-tail and native-order work in other import paths. That is real progress and is not challenged here. The current `seriesJetApply` implementation still has an independent native request, its own structural admission checks, and its own coefficient accessor. The shortened-oracle test reaches that path through the public `SeriesObservable` API. Extending C06 with this witness is more accurate than declaring the earlier fix nonexistent or reopening every native import. [10, 4]

6 N03: backend defaults do not control dispatch

Priority: high. Native default-versus-explicit behavior reproduced.
Source: `expansionDispatch` in `NativeCompatibility.wl`.
Effect: configured defaults do not select the advertised engine, including the default intended to prohibit native fallback.

6.1 An order-convention witness

Evaluate

```
SetOptions[AsymptoticAnalysis‘AsymptoticExpansion,
  "Backend" -> "Series"];
s = AsymptoticAnalysis‘AsymptoticExpansion[Exp[x], {x,0,2}];
{s["Kind"], s["Expression"]}
(* observed: {"Forward", 1+x} *)
```

The equivalent explicit-selector request returns

```
s = AsymptoticAnalysis‘AsymptoticExpansion[
  Exp[x], {x,0,2}, "Backend" -> "Series"];
{s["Kind"], s["Expression"]}
(* observed: {"Native", 1+x+x^2/2} *)
```

The configured option has not become the request's default. This conflicts with the purpose of `SetOptions`: it changes default option values for subsequent uses of the relevant symbol. [21]

6.2 A package-only policy witness

The opposite default demonstrates a more consequential distinction:

```
SetOptions[AsymptoticAnalysis‘AsymptoticExpansion,
  "Backend" -> "Package"];
s = AsymptoticAnalysis‘AsymptoticExpansion[Exp[I x], {x,0,2}];
s["Kind"]
(* observed: "Native" *)
```

With `"Backend"->"Package"` explicitly present in that call, the observed result is `Failure["InexactInput", ...]` because the package analytic path rejects the complex constant. The exact failure tag is broader in wording than the reason relevant here, but the key contrast is unambiguous: the default package-only policy permits a native result while the explicit policy does not.

This is not a claim that the native complex result is mathematically wrong. It is a failure to honor an engine/admission policy the user configured. A caller relying on the default to obtain analytic package objects can receive a Native object instead. A later arithmetic operation may then fail for a reason far removed from the original call, obscuring the actual cause.

6.3 The hardcoded fallback

The dispatcher scans held trailing option containers for explicit selector values. Its no-selector branch is

```
backend = If[values === {}, Automatic, ReleaseHold[First[values]]];
```

Thus the published option table is not consulted when the selector is omitted. Everything after this line can route correctly for the value it received and still implement the wrong public default. [6]

The minimal candidate replacement is

```
backend = If[values === {},
  OptionValue[AsymptoticExpansion, {}, "Backend"],
  ReleaseHold[First[values]]];
```

This preserves explicit-selector precedence and resolves the canonical constructor's default exactly when no explicit selector was found. The native prototype using this replacement returned `"Native"` and $1 + x + x^2/2$ for the first witness.

6.4 Alias defaults and delayed evaluation need a stated policy

`AsymptoticExpand` is a held alias that forwards to `AsymptoticExpansion`. The initial option tables are copied, but an alias with its own mutable `Options` table raises an additional policy question: which symbol owns defaults after `SetOptions` changes one table? The narrow patch fixes the canonical constructor's default; it does not synchronize independently changed alias defaults. [6]

The cleanest small policy is to make the canonical constructor the single owner of defaults and state that the alias shares them. Its option presentation should then avoid implying that an independently changed alias table controls execution. An alternative is to give the alias an explicit default-resolution layer before forwarding. Either is defensible; a stale mutable copy that looks authoritative but is ignored is not a good long-term interface.

The regression matrix should also include delayed backend defaults, nested option lists, explicit overrides, and calls whose source has a counter. A backend-selection fix must not replay a source expression merely to discover an option. The held dispatch infrastructure already goes to considerable effort to preserve that separation, so the repair should stay within it rather than replacing the entire entry path. [6, 7]

7 Smaller API and performance findings

7.1 N04: zero-order differentiation ignores an explicit cutoff

For an exact ordinary input,

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  x, {x,0,5}, "Backend"->"Package"];
r = AsymptoticAnalysis`SeriesDifferentiate[s, 0, "Cutoff"->1];
{r === s, r["Expression"], r["Remainder"]}
(* observed: {True, x, 0} *)
```

Returning s is correct as a zeroth derivative. The inconsistency is that the explicit exclusive cutoff has no effect. Applying cutoff one to the ordinary x block would omit that block and retain an $O(x)$ remainder. The current

`seriesDerivative` has an early `If[n===0,Return[s,...]]` before the ordinary cutoff is applied. [4]

This should be recorded as a low-priority API policy issue, not as an incorrect asymptotic formula. A reasonable policy is that order zero performs no differentiation but still applies an explicit output cutoff. Under that policy, the early return should delegate to `SeriesTruncate` when a cutoff is present. Another coherent policy is to reject a cutoff for order zero. The current silent no-op on the cutoff is the least informative choice.

An initial suspicion that this early return also bypassed `MaxTerms` validation was tested and disproved. With `"MaxTerms"->0`, the observed result was `Failure["InvalidOption",...]`. The report therefore does not claim a resource-validation bypass. This control is included because it prevents a plausible but incorrect extension of the source-level finding.

7.2 N05: unknown symbolic options can evade option admission

The following explicit package request produced a normal forward object:

```
AsymptoticAnalysis`AsymptoticExpansion[
  Exp[x], {x,0,2}, MadeUpOption->123, "Backend"->"Package"]
(* observed Kind: "Forward"; Expression: 1+x *)
```

It also emitted repeated `OptionValue::nodef` warnings. This is therefore *success with warnings*, not silent success. In code that treats `FailureQ` as its error boundary, however, the request appears successful and the unsupported option is effectively ignored.

The source explains why this bypasses the new package-option check:

```
nativeSpecificationQ[
  HoldComplete[(Rule | RuleDelayed)[key_Symbol, _]]] :=
  ! MemberQ[First /@ Options[AsymptoticExpansion], Unevaluated[key]];
```

Any symbolic rule whose name is not a known option is treated as an expansion specification. Then `nativeRequestOptionKeys` excludes specifications from its option scan, so the unknown symbol never reaches the explicit package exclusive-option check. An unknown string key does not match that symbolic specification rule and is subject to the check. The string-path consequence follows from the inspected source; the bundled native observation is the symbolic-path call above. [6]

This grammar is also unstable under changes to option tables: the same held rule can change syntactic role when an unrelated option name becomes known. Role classification should primarily follow the public argument grammar and position, not the complement of the current option-name set. This is especially important in a wrapper whose native coverage deliberately evolves over time.

A suitable small refactoring parses a request into an unevaluated source, ordered expansion specifications, wrapper options, and native options. In an explicit package call, one legal package specification is recognized in its allowed position; remaining rules are options and unknown names receive a structured failure. Explicit native modes can retain broader native argument forms without pretending that the package grammar has admitted them.

This is not a proposal to replay or aggressively evaluate held options. Evaluation policy and syntactic role discovery are separate concerns. The current option-container traversal already preserves several subtle cases; those tests should be retained while role classification is made explicit.

7.3 N06: eager native projection is measurable extra work

The native wrapper computes

```
result = Block[{$Assumptions = ambient}, ReleaseHold[call]];
normal = Normal[result];
```

before returning the object, storing both the native payload and the projected expression. This is intentional in the current result contract, but it means that callers requesting only `s["NativeResult"]` pay for the projection anyway. [6, 7]

An instrumented native constant makes the additional evaluation visible:

```
normalCalls = 0;
normalProbe /: Normal[normalProbe] := (++normalCalls; 7);
direct = Series[normalProbe, {x,0,2}];
s = AsymptoticAnalysis`AsymptoticExpansion[
  normalProbe, {x,0,2}, "Backend"->"Series"];
{normalCalls, s["NativeResult"], s["Expression"]}
(* observed: {1, normalProbe, 7} *)
```

The stored native result is preserved; this is not a claim of payload corruption. The observation establishes one extra custom-normalization call before the expression property is requested. It does not establish any particular wall-clock cost, physical-memory duplication, or asymptotic speedup. `Normal` is a general conversion operation, not a universally free removal of an `0` term. [20]

7.4 A compatibility-conscious optimization

A lazy or native-only projection mode could avoid this work for clients that consume native payloads. It should be an explicit design choice, because moving `Normal` to a later time can change when user-defined normalization code runs and which ordinary definitions it sees. Silently changing eager semantics to lazy semantics would exchange a performance concern for an evaluation-semantics concern.

A useful first experiment compares direct native evaluation, eager wrapper construction, and a prototype native-payload-only wrapper. Record the number of projection calls, time spent in projection, result leaf counts, and actual peak memory separately. Include an ordinary `SeriesData` control, an array of results, a large nested result, and a custom-normalization payload. Do not infer physical duplication simply by adding two `ByteCount` values, since expression sharing and retained references can affect actual storage.

A native-only mode also needs a clear response for `Normal[s]`, numerical application, and `s["Expression"]`. Missing projection is not exact zero, and not an analytic remainder. The existing refusal of native analytic arithmetic must remain unchanged. This focused opportunity is distinct from the already documented sparse-to-dense analytic exporter problem.

8 Candidate changes and their validation boundary

8.1 What is included

The accompanying patch specification has three edits affecting two modular files. It changes backend-default resolution in `NativeCompatibility.wl`, adds a bounded necessary continuity screen to `SeriesOperatio`

`ns.wl`, and checks the native exclusive order before the generic observable coefficient accessor is used. The continuity screen deliberately refuses uncertain or discontinuous cases rather than introducing an unreviewed sided observable implementation.

`stage_patch.py` reads an existing checkout and writes only the changed modular files into a new external staging directory. Every source anchor must match exactly once. All transformations are validated before any output file is created. Missing, duplicated, or already-patched anchors cause failure; the input checkout is never modified. Exact anchors protect the specific edit sites, not the identity of every other file in a checkout.

The standalone-copy helper is for focused experiments. Actual development should apply reviewed changes to modular sources and regenerate the standalone with the repository's existing builder. No generated full package is included in this bundle, and no repository file was modified during this review.

8.2 What was actually checked

The independent Python program ran fourteen test methods successfully. They verify exact rational counterexample ratios, the native-tail composition-order model, simple order-convention examples, and failure behavior of the patch staging transformation on synthetic fixtures. These checks are small enough to inspect directly and require only the Python standard library.

A temporary patched standalone was also loaded in the native evaluator. Its three successful focused observations were rejection of the retained-left `FractionalPart` witness, preservation of the ordinary `Sin` expansion, and correct selection of the configured native `Series` backend. The prototype used the same guard conditions and default-resolution mechanism as the supplied patch, with abbreviated diagnostic strings and formatting.

The complete packaged WLT file, the exact emitted modular patch in a local checkout, the repository builder, and the full upstream suite were *not* run. In particular, the shortened-oracle rejection is specified in the packaged regression tests and justified by the added condition, but it was not part of the successful three-control patched native observation. This distinction is preserved in the evidence files.

8.3 Why the patch is intentionally narrow

Replacing `h[c]` by a native zeroth coefficient everywhere would fix the retained-left example but could still select the wrong side for a pure remainder. A general sided implementation therefore needs a separate admission record. Conversely, a continuity screen alone does not verify a Taylor error of arbitrary order. The supplied hardening patch closes the demonstrated jump mechanism conservatively and prevents the particular native-order promotion; it is not presented as a complete general analyticity checker.

The zero-derivative cutoff and unknown-option issues are supplied as separate proposed-policy tests, not silently bundled into the correctness patch. The native-projection optimization is not patched at all, because its evaluation timing is part of the existing contract. This separation keeps a small correctness change from accidentally becoming a wide API revision.

9 Development priorities beyond the existing reviews

9.1 First: make oracle results carry their own information boundary

The immediate development target is a checked internal Taylor-oracle result, not a new global abstraction for every scale. Its minimum fields are the coordinate, selected approach, finite coefficient support, exclusive returned order, limiting constant, and origin of the regularity premise. Coefficient lookup must distinguish unavailable values

from known zeros.

The same small importer should serve the generic observable branch and other ordinary Taylor-consuming adapters where the contracts really match. Existing specialized paths with different asymptotic scales should not be forced through it merely for architectural uniformity. In particular, an ordinary scalar Taylor importer is not a parser for an arbitrary nested native expression or a Gamma inverse coefficient algebra.

Its acceptance tests should vary the *returned* order independently of the requested order. A handler can return a smaller prefix, a larger prefix, an exact constant, a zero prefix with a nonzero remainder, or a native result with conditions. The consumer's response should depend on the evidence received, not on an assumption that every request was completely fulfilled. N02 gives a compact regression for this distinction.

9.2 Second: implement sided observable germs without endpoint substitution

A narrow useful extension supports observables possessing regular one-sided expansions at a finite endpoint. It need not begin with arbitrary discontinuous functions. Start with a small class where the branch boundary and sided polynomial are straightforward, then verify that the entire inner germ remains on the chosen side.

The algorithm is: determine whether the inner germ is exactly constant; otherwise establish its finite center and, when needed, a strict eventual side; acquire a native or dedicated sided expansion; preserve its actual zeroth coefficient; transport the native tail and the input uncertainty independently; finally assemble the output and record the sided contract. If the sign is unknown, return a structured request for more input precision or a compatible two-sided contract rather than silently choosing the positive side.

This gives a concrete positive result for the N01 witness: with the $-x$ block retained, return $1 - x$ with a justified error. With only $1 + O(x)$ and no retained source-side evidence, decline or refine the source before choosing a sided observable. For an exactly constant inner one, return `FractionalPart[1]` equal to zero. All three outputs can be correct; the input evidence distinguishes them.

9.3 Third: normalize request roles and default ownership

The native dispatcher should have one explicit request-role pass and one explicit default-resolution policy. These are small, testable boundaries. A parsed request need not eagerly evaluate its source; held syntax can still be stored in separate source, specification, and option fields. The role parser must not use "not a known option" as a general definition of "variable specification" in positions where an option is expected.

Canonical defaults, alias defaults, explicit selectors, nested option containers, and delayed option values should be tested independently of mathematical expansion. A backend default is a routing input, not decorative metadata. Source-side counters and delayed-option counters can detect accidental replay while the matrix checks which head actually received the request.

The expected output should include whether the choice was explicit or defaulted, which symbol supplied a default, and the selected engine. Such a record would make the original N03 error immediately visible without inspecting the degree of an exponential polynomial. This recommendation extends the existing native selection metadata at one specific missing point rather than proposing another broad diagnostics framework.

9.4 Fourth: add germ-specific metamorphic tests

Several invariants can test broad classes of expressions more effectively than a growing list of unrelated examples.

Endpoint-value invariance. For an eventually separated inner germ, modify only the outer endpoint value. The composition must remain unchanged near the limit. Keep an exact-constant negative control, where the point

value must be used.

Oracle weakening. Replace a truthful oracle by one returning a shorter truthful prefix. The consumer may return a weaker result or refuse, but must not report more information than it did with the stronger oracle. This is a local boundary test, not a general demand-planning benchmark.

Side reflection. Reflect a known inner displacement and the outer sided chart together. Reconstructed constants and powers should transform consistently. For `FractionalPart` around one, the left and right constants provide a deliberately asymmetric test.

Default-explicit equivalence. In a controlled environment, setting a canonical default and omitting that option should select the same engine as an otherwise equivalent explicit option. Compare held-call effects as well as finite expressions, since the option can determine which evaluator sees the source first.

Identity operation with output policy. For order-zero differentiation, separate the identity property from any requested truncation. A no-cutoff call should preserve the original object; a cutoff call should follow the chosen published cutoff policy. Include invalid-option controls so a convenient early return does not become an unchecked path.

These are finite acceptance properties with clear failure witnesses. They do not require solving a general equivalence or analyticity problem, and they do not duplicate the existing proposal for a full release gate.

9.5 Fifth: measure projection and admission costs separately

The interim continuity screen introduces up to two native limit computations per generic observable admission, bounded together by its time constraint. That is an intentional potential availability cost. A benchmark should report how much time is spent acquiring coefficients, screening continuity, composing jets, and normalizing output, rather than attributing the total to one "series operation."

Repeated identical center/head/assumption checks within a single request are a possible narrowly scoped optimization, provided the request has a well-defined ordinary-definition context. Reusing a result across unrelated sessions or changed definitions is a different and unsafe policy. The immediate objective is to expose the cost of the new proof obligation, not to mask it with a global memoization table.

For N06, first measure whether eager normalization is material in representative workloads. Only then choose an explicit native-only or deferred-projection interface. The useful performance result is a measured reduction at an unchanged semantic contract, or a clearly advertised new contract with a measured cost benefit. No such speedup is claimed by this review.

9.6 Acceptance order

The most direct sequence is to land the backend-default correction with isolated routing tests; land the native-order guard with the shortened-oracle regression; add conservative observable admission for the reproduced jump cases; then implement a richer sided-germ path behind explicit tests. The smaller cutoff and option-grammar policies can be reviewed independently. Projection changes should follow measurement and an evaluation-timing decision.

This sequence does not imply that previously recorded high-priority defects are unimportant. It is the incremental work identified here. The existing register remains the source of truth for the earlier dense-export, assumption, precision, parameter-capture, certificate, and flat-sector work. Replacing that register with another duplicated roadmap would make progress harder to assess.

10 Conclusion

The repository's strongest idea is the retained asymptotic object: a finite approximation with a coordinate, a meaning for its omitted part, and enough provenance to support subsequent work. That idea benefits from native Wolfram capabilities rather than competing with them at the level of a function-name checklist.

The new failures occur when information is reinterpreted at a boundary. A limiting coefficient is replaced by a point value; an unknown native coefficient is replaced by zero; a configured backend default is replaced by `Automatic`. In each case the replacement is small in code but large in meaning. The accompanying witnesses, candidate edits, and local invariants make those replacements directly testable.

The immediate recommendation is therefore precise: preserve the native oracle's actual constant and order under an admitted germ contract, and preserve the user's actual backend choice through default resolution. Broader sided observables and performance work can then be developed on top of those repaired boundaries without weakening the analytic/native/certificate distinctions the package already gets right.

A Reproduction and bundle contents

The archive is organized as follows:

```
article/article.tex              article/article.pdf
code/patch_spec.json             code/stage_patch.py
code/check_independent.py        code/native_characterizations.wl
code/focused_candidate_tests.wlt code/proposed_policy_tests.wlt
code/apply_candidate_to_standalone.wl
code/native_projection_probe.wl
evidence/native_observations.json
evidence/novelty_ledger.json      evidence/source_coverage.json
evidence/independent_checks.json and .txt
README.md                       LICENSE-AUDIT.txt
UPSTREAM-NOTICE.txt              build.sh
```

No checksum files or font files are included.

To run the independent checks:

```
python code/check_independent.py
```

To reproduce baseline native observations from a local standalone:

```
wolframscript -file code/native_characterizations.wl /path/AsymptoticAnalysis.wl
```

To stage the candidate modular edits without modifying a checkout:

```
python code/stage_patch.py /path/to/Asymptotic /path/to/new-staging-directory
```

Review the staged diff before applying it to a separate development checkout, regenerate its standalone with the existing repository builder, and load that candidate before using `TestReport` on `focused_candidate_tests.wlt`. The policy tests are intentionally separate and are not implemented by the candidate hardening patch.

B Coverage and novelty crosswalk

Source / activity	Scope and limitation
`NativeCompatibility.wl`	Close inspection of the dispatcher, option/specification classification, fallback policy, and native wrapper.
`SeriesOperations.wl`	Close inspection of representation conversion, observable import, composition, truncation, differentiation, and the start of refinement handling.
`SeriesArithmetic.wl`	Selected central arithmetic, held normalization, regular-operand conversion, and native refusal paths.
`InverseCertificates.wl`	Nearly complete inspection of interval arithmetic, domain checks, root enclosure, and adaptive accuracy handling; no new certificate defect asserted.
`InverseFunctionExpressions.wl`	Selected held entry, domain, branch and observable-condition paths.
Main kernel and Gamma inverse	Public contracts, failure/assumption context, current module load list, and ordered Gamma/Barnes inverse construction.
Documentation and review register	Current README, module map, native result contract, review indices, complete maintained finding crosswalk, and selected original review discussions.
Native execution	Focused baseline counterexamples and controls; three temporary-patch mechanism controls. No full suite, release gate, or native performance benchmark.
Independent execution	Fourteen exact mathematical and synthetic patch-fixture test methods; no inference of package correctness from Python alone.

N01 extends the existing broad C13/C16 obligations with a built-in sided-constant counterexample and a reconstruction-specific fix. N02 extends C06 to a separate unchecked observable consumer. N03 and N05 identify concrete defects in the newer dispatcher rather than repeating B01–B03's broad coverage objective. N04 is a small operation-policy inconsistency distinct from the already recorded lower-cutoff `SeriesRefine` behavior. N06 concerns the newer native wrapper's projection cost, not the old dense native exporter.

The lexically searched root TeX files contained no occurrences of `FractionalPart`, `SetOptions`, `zero-order`, `zeroorder`, or `shortened`. That observation is useful but not conclusive: included TeX sections and different terminology can evade a root-file keyword search. The crosswalk and the actual overlap analysis above are the substantive basis for the incremental classification.

C What the report deliberately does not repeat

The maintained register already covers dense rational-lattice export, nonlinear remainder frontiers, pure-remainder fractional-power guards, ambient assumption capture, semantic weight collisions, parameter capture in composition, resource accounting, inverse multi-index enumeration, factorization cost, certificate adaptivity, flat-grade preservation, saved-object migration, and major future transseries extensions. Those entries remain relevant, but they are not counted as new findings here. [10]

Similarly, a broad call for integration, complex sectors, uniform parameter asymptotics, multivariate implicit

systems, formal proof export, or native release automation would duplicate already recorded development proposals. The present roadmap is limited to the specific observable, oracle, request, and projection boundaries exposed by the new evidence.

References

[1] Asymptotic repository, current README, pinned revision `6687962`.

[2] Asymptotic repository, main modular kernel, especially public definitions, assumption scope, and module loading.

[3] Asymptotic repository, kernel architecture map.

[4] Asymptotic repository, series operations, especially `seriesJetApply`, `SeriesObservable`, `seriesDerivative`, and representation conversion.

[5] Asymptotic repository, ordinary arithmetic and held normalization.

[6] Asymptotic repository, native compatibility, especially `expansionDispatch`, `nativeSpecificationQ`, and `nativeExpansion`.

[7] Asymptotic repository, native expansion result contracts.

[8] Asymptotic repository, exact rational inverse certificates.

[9] Asymptotic repository, Gamma/Barnes inverse model and constructor.

[10] Asymptotic repository, consolidated code-review implementation status and complete crosswalk.

[11] Asymptotic repository, external review index, with linked wave-1 and wave-2 indices.

[12] Existing review 4, source-pinned technical audit; especially the additional proof obligations R01–R03.

[13] Existing review 11, contract audit; regularity-admission discussion and the `Analytic->False` proposal.

[14] Wolfram Research, *Series*, official documentation, accessed 9 September 2026. https://reference.wolfram.com/language/ref/Series.html

[15] Wolfram Research, *Asymptotic*, official documentation. https://reference.wolfram.com/language/ref/Asymptotic.html

[16] Wolfram Research, *SeriesData*, official documentation. https://reference.wolfram.com/language/ref/SeriesData.html

[17] Wolfram Research, *InverseSeries*, official documentation. https://reference.wolfram.com/language/ref/InverseSeries.html

[18] Wolfram Research, *ComposeSeries*, official documentation. https://reference.wolfram.com/language/ref/ComposeSeries.html

[19] Wolfram Research, *AsymptoticSolve*, official documentation; including native series output and multivariate scope. https://reference.wolfram.com/language/ref/AsymptoticSolve.html

[20] Wolfram Research, *Normal*, official documentation. https://reference.wolfram.com/language/ref/Normal.html

[21] Wolfram Research, *SetOptions*, official documentation. https://reference.wolfram.com/language/ref/SetOptions.html

[22] Wolfram Research, *FractionalPart*, official documentation. https://reference.wolfram.com/language/ref/FractionalPart.html

[23] Wolfram Research, *FunctionAnalytic*, official documentation. https://reference.wolfram.com/language/ref/FunctionAnalytic.html

[24] Wolfram Research, *FunctionDomain*, official documentation. https://reference.wolfram.com/language/ref/FunctionDomain.html

[25] Wolfram Research, *AsymptoticLess*, official documentation. https://reference.wolfram.com/language/ref/AsymptoticLess.html

[26] Wolfram Research, *AsymptoticIntegrate*, official documentation. https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html

[27] Wolfram Research, *AsymptoticSum*, official documentation. https://reference.wolfram.com/language/ref/AsymptoticSum.html

[28] NIST Digital Library of Mathematical Functions, Section 5.11, *Asymptotic Expansions*. https://dlmf.nist.gov/5.11

[29] NIST Digital Library of Mathematical Functions, Section 5.17, *Barnes' G-Function*. https://dlmf.nist.gov/5.17